

# AutoTrack

Auto Parts Inventory & Distribution System

---

Technology Stack Decision Framework

CS 490 Capstone Project

# Project Overview

---

## WHAT IT IS

Inventory and operations system for auto parts management

Target: Small auto shops, repair networks, parts warehouses

## WHAT IT ISN'T

Full-scale ERP or manufacturer-level logistics platform

## CORE CAPABILITIES

- Track parts by SKU, compatibility, quantity
- Manage requests and fulfillment workflows
- Monitor stock levels with alerts
- Track part movement lifecycle
- Analytics on usage and demand

# Module Architecture

---

## MODULE\_01

### Inventory Management

Part catalog, SKU tracking, compatibility, location, quantity updates

## MODULE\_02

### Request & Fulfillment

Submission, approval, fulfillment tracking, pickup/delivery status

## MODULE\_03

### Supplier & Restocking

Supplier records, restock thresholds, alerts, order history

## MODULE\_04

### Operations Dashboard

Real-time stock overview, low-stock indicators, high-demand alerts

## MODULE\_05

### Analytics & Reporting

Usage trends, turnover rates, demand summaries, forecasting

Primary modules (blue) | Reporting modules (green)

# Frontend Framework Options

| Framework | Industry Use                                       | Job Market                                 | Best Use Cases                                            | Link                                        |
|-----------|----------------------------------------------------|--------------------------------------------|-----------------------------------------------------------|---------------------------------------------|
| React     | Very High - Facebook, Netflix, Airbnb, 220k+ stars | Excellent - Most demanded, 40% of postings | Complex dashboards, SPAs with real-time data, large teams | <a href="https://react.dev">react.dev</a>   |
| Vue       | High - Alibaba, GitLab, Adobe, 210k+ stars         | Good - Growing demand, 15% of postings     | Rapid prototyping, small-to-medium apps, solo developers  | <a href="https://vuejs.org">vuejs.org</a>   |
| Svelte    | Growing - The New York Times, 79k+ stars           | Emerging - 5% of postings, growing fast    | Performance-critical apps, minimal bundle size needs      | <a href="https://svelte.dev">svelte.dev</a> |
| Angular   | High - Google, Microsoft, Forbes, 96k+ stars       | Good - Stable demand, 20% of postings      | Enterprise apps, strict architecture, large teams         | <a href="https://angular.io">angular.io</a> |

Job market percentages based on 2024 developer surveys • All options are open-source

# Backend Framework Options

| Framework  | Industry Use                                  | Job Market                          | Best Use Cases                                           | Link                                                                      |
|------------|-----------------------------------------------|-------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------|
| Express.js | Very High - Uber, IBM, Accenture. 15k req/sec | Excellent - 35% of Node.js jobs     | Full-stack JS teams, real-time features, microservices   | <a href="https://expressjs.com">expressjs.com</a>                         |
| FastAPI    | Growing - Microsoft, Netflix ML. 25k req/sec  | Strong - 25% of Python API jobs     | Data science integration, ML APIs, rapid API development | <a href="https://fastapi.tiangolo.com">fastapi.tiangolo.com</a>           |
| Flask      | High - Pinterest, LinkedIn. 10k req/sec       | Good - 20% of Python web jobs       | Simple APIs, prototypes, learning Python web dev         | <a href="https://flask.palletsprojects.com">flask.palletsprojects.com</a> |
| NestJS     | Growing - Adidas, Roche. 20k req/sec          | Emerging - 15% of Node jobs, rising | Large teams, complex business logic, TypeScript fans     | <a href="https://nestjs.com">nestjs.com</a>                               |

Req/sec = requests per second benchmark • Job percentages from 2024 Stack Overflow survey

# Database Options

| Database   | Industry Use                           | Job Market                         | Best Use Cases                                          | Link                                                |
|------------|----------------------------------------|------------------------------------|---------------------------------------------------------|-----------------------------------------------------|
| PostgreSQL | Very High - Apple, Instagram, Spotify  | Excellent - 30% of DB jobs, rising | Complex joins, ACID compliance, inventory relationships | <a href="https://postgresql.org">postgresql.org</a> |
| MongoDB    | High - eBay, EA, Adobe                 | Good - 20% of NoSQL jobs           | Flexible schemas, rapid iteration, nested documents     | <a href="https://mongodb.com">mongodb.com</a>       |
| MySQL      | Very High - Facebook, Twitter, YouTube | Excellent - 35% of SQL jobs        | Web apps, high read workloads, simpler than PostgreSQL  | <a href="https://mysql.com">mysql.com</a>           |
| SQLite     | High - Android, iOS apps, browsers     | Moderate - 10% of embedded jobs    | Prototyping, single-server apps, embedded databases     | <a href="https://sqlite.org">sqlite.org</a>         |

Job market data from 2024 database surveys • Companies listed use each DB in production

# Barcode/QR Scanning Options

| Hardware Option            |                                  | Difficulty                                          | Cost Range             | Best For                                                |                                                                                 |
|----------------------------|----------------------------------|-----------------------------------------------------|------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------|
| USB Barcode Scanner        |                                  | Easy - Plug & play, acts as keyboard                | \$30-\$150             | Fixed stations, high-volume scanning, reliable input    |                                                                                 |
| Mobile Device Camera       |                                  | Easy - Uses existing hardware                       | \$0 (existing devices) | Mobile workflows, flexible scanning, no extra equipment |                                                                                 |
| Dedicated Handheld Scanner |                                  | Moderate - Requires integration                     | \$200-\$500            | Warehouse use, rugged environments, long battery life   |                                                                                 |
| Library                    | Format Support                   | Frontend Integration                                |                        | Backend Needed?                                         | Link                                                                            |
| quagga2                    | 1D barcodes (UPC, EAN, Code 128) | React/Vue/Angular: Easy npm install, works with any |                        | No - Client-side only                                   | <a href="https://github.com/ericblade/quagga2">github.com/ericblade/quagga2</a> |
| jsQR                       | QR codes only                    | React/Vue/Svelte: Very easy, pure JS, <5KB          |                        | No - Client-side only                                   | <a href="https://github.com/cozmo/jsQR">github.com/cozmo/jsQR</a>               |
| zxing                      | 1D & 2D barcodes (all formats)   | All frameworks: Easy, most comprehensive            |                        | Optional - Can use backend for heavy processing         | <a href="https://github.com/zxing-js/library">github.com/zxing-js/library</a>   |

Consider hardware costs vs. implementation complexity when choosing scanning solution

# Visualization Libraries

| Library                        | Industry Use                                          | React Integration                          | Vue/Angular Integration                        | Link                                  |
|--------------------------------|-------------------------------------------------------|--------------------------------------------|------------------------------------------------|---------------------------------------|
| <a href="#">Chart.js</a>       | Very High - 60k GitHub stars, used by IBM, Salesforce | Good - react-chartjs-2 wrapper available   | Good - Official Vue/Angular wrappers available | <a href="#">chartjs.org</a>           |
| <a href="#">Recharts</a>       | High - 20k GitHub stars, React-specific library       | Excellent - Built for React, declarative   | Not recommended - React-only library           | <a href="#">recharts.org</a>          |
| <a href="#">D3.js</a>          | Very High - 108k GitHub stars, NYT, FiveThirtyEight   | Good - Many React wrappers (Nivo, Victory) | Good - Framework-agnostic, full control        | <a href="#">d3js.org</a>              |
| <a href="#">Apache ECharts</a> | High - 57k GitHub stars, Alibaba, Baidu, Tencent      | Good - echarts-for-react wrapper           | Excellent - Official Vue/Angular support       | <a href="#">echarts.apache.org</a>    |
| <a href="#">Plotly.js</a>      | High - 16k GitHub stars, scientific/technical focus   | Good - react-plotly.js wrapper             | Good - Framework-agnostic, scientific charts   | <a href="#">plotly.com/javascript</a> |

GitHub stars = developer bookmarks (popularity metric) • Industry usage from 2024 surveys • All have TypeScript support



# Key Questions to Address

---

## Team Experience

What languages and frameworks is the team already comfortable with?

## Development Velocity

Do we optimize for rapid development or production-ready scalability?

## Data Complexity

How complex are the relationships between parts, suppliers, and requests?

## Module Integration

How will the five modules communicate? REST API? GraphQL? Shared database?

## Authentication Strategy

JWT tokens? Session-based? OAuth? How do we handle role-based access?

## Testing & Deployment

Unit tests? Integration tests? Containerization? CI/CD pipeline?

# Next Steps

---

1

## Team Survey & Skills Inventory

Assess current experience with proposed technologies to inform final stack selection

2

## Architecture Design Session

Map out module interfaces, API contracts, and data flow between components

3

## Proof-of-Concept Sprint

Build a minimal working example with chosen stack to validate integration approach

4

## Module Assignment & Development Kickoff

Assign modules based on interests and begin parallel development

Timeline: Weeks 1-2 for decisions, Week 3 for POC, Week 4+ for development